AI Quality Pipeline — Architecture, Flow, and Impact

Overview

The AI pipeline is a post-calculation analysis layer that takes completed lighting design results from LuxScaleAI and applies AI reasoning to detect issues, score quality, and suggest improvements. It runs **after** the core calculation engine — it does not change how fixtures are calculated, but helps interpret and improve those results over time.

Part 1 — Pipeline Architecture

Components

```
luxscale/
├── gemini_manager.py     — Main orchestrator: accounts, waterfall, config
├── ollama_manager.py     — Local model interface (Ollama REST API)
├── ai_routes.py          — Flask endpoints (/api/ai/analyze, /approve-fix, /status, /account)
└── gemini_config.json    — Live config: keys, model, priority, quota counters
```

Configuration file: `gemini_config.json`

All behavior is controlled by this single file — no code changes needed to switch models, accounts, or priority:

```
{
  "accounts": [
    { "name": "account_1_free", "api_key": "AIzaSy...", "daily_limit": 15, "used_today": 3, "enabled":
    { "name": "account_2_free", "api_key": "AIzaSy...", "daily_limit": 15, "used_today": 0, "enabled":
    { "name": "account_3_free", "api_key": "", "daily_limit": 15, "enabled": false },
    { "name": "account_5_paid", "api_key": "", "daily_limit": 800, "enabled": false }
  ],
  "model": "gemini-3.1-flash-lite-preview",
  "ollama_priority": true,
  "min_confidence": 0.6,
  "timeout_seconds": 15
}
```

Field	Effect
`model`	Which Gemini model to use across all accounts
`ollama_priority`	`true` = Ollama first (saves tokens), `false` = Gemini first (better quality)
`min_confidence`	Minimum confidence score to accept a result (0.0–1.0)
`daily_limit`	Max requests per day per account (auto-resets at midnight)
`used_today`	Auto-incremented counter, never manually edit

Part 2 — Waterfall Logic

Priority modes

`ollama_priority: true` (recommended while all accounts are free tier):

```
Request → Ollama (local, free, unlimited) → Gemini acc 1 → Gemini acc 2 → ... → Snapshot → Default
```

`ollama_priority: false` (recommended with paid Gemini account):

```
Request → Gemini acc 1 → Gemini acc 2 → ... → Ollama (local backup) → Snapshot → Default
```

Waterfall decision tree

```
analyze_lighting_result(payload)
│
├─ ollama_priority=true?
│   ├─ YES → try Ollama
│   │         ├─ available + valid JSON → return (source: ollama:local)
│   │         └─ unavailable/failed → fall through to Gemini
│   └─ NO  → skip to Gemini first
│
├─ Try Gemini accounts in order
│   ├─ Account has quota + valid key?
│   │   ├─ YES → call API
│   │   │         ├─ 429 rate limit → SKIP KEY → next account
│   │   │         ├─ 400/401/403 bad key → SKIP KEY → next account
│   │   │         ├─ Valid JSON, confidence ≥ min_confidence → return (source: gemini:account_name)
│   │   │         └─ Low confidence / parse error → next account
```

```
│   │   └─ NO → skip (log reason)
│   └─ All accounts exhausted
│
├─ [if ollama_priority=false] try Ollama as backup
│
├─ Load snapshot (last approved analysis)
│   ├─ Exists → return (source: snapshot, snapshot_saved_at: ...)
│   └─ Not found → fall through
│
└─ Return safe default (source: default)
    quality_score: 50, confidence: 0, issues: []
```

Quota protection

Each account tracks daily usage in `gemini_config.json`:

- `used_today` increments after every successful call
- `reset_date` stores the last reset date — auto-resets to 0 on a new calendar day
- `daily_limit` is set conservatively (15 for free tier with 20 RPD limit = 25% buffer)
- Account 5 (paid) has `daily_limit: 800` = 80% of a typical paid plan

Part 3 — What the AI Analyzes

Input to the AI

The prompt is built from the study payload — specifically the first result row:

```
Space: Office  Sides: [10, 8, 10, 8]m  H: 3.2m
Required: Em=500lx  Uo=0.6
Got: Em=487lx(LOW) U0=0.54(LOW) Fixtures:24
```

Expected JSON output

```
{
  "confidence": 0.9,
  "quality_score": 65,
  "issues": [
    {
      "severity": "high",
      "field": "Average Illuminance (Em)",
      "description": "Calculated lux is 2.6% below required 500 lux",
      "suggested_fix": "Increase fixture count to 26 or use higher efficacy"
```

```
    },
    {
      "severity": "medium",
      "field": "Uniformity (U0)",
      "description": "U0=0.54 is below required 0.6",
      "suggested_fix": "Adjust spacing or use wider beam angle fixtures"
    }
  ],
  "suggestions": [
    "Consider SC triproof for better uniformity in this room size",
    "Increase mounting height if ceiling allows"
  ],
  "summary": "Design narrowly misses both lux and uniformity targets. Minor adjustments to fixture cou
}
```

Validation before accepting

Before any result is used:

1. Must be parseable JSON (three-attempt parser: strip fences → find `{...}` block → extract outermost braces)
2. `confidence` field must be ≥ `min_confidence` (default 0.6)
3. Missing fields are auto-filled with safe defaults (empty arrays, 50 score, etc.) rather than rejected

---

Part 4 — Snapshot System (Fallback Safety)

When the user clicks “Approve & Save as Baseline” in the UI, the current analysis is written to `gemini_snapshot.json`:

```
{
  "saved_at": "2026-04-04T16:50:32Z",
  "analysis": {
    "confidence": 0.9,
    "quality_score": 70,
    "issues": [...],
    "suggestions": [...],
    "summary": "..."
  }
}
```

If all AI sources fail in future requests, this snapshot is returned with `source: snapshot`. This ensures the system always has a meaningful response even with zero connectivity — it falls back to the last known-good analysis rather than an empty default.

Part 5 — Before and After the AI Pipeline

Before: Raw calculation output only

Without the AI pipeline, a user receiving results sees only the raw numbers:

```
Average Lux: 487.3
U0_calculated: 0.5421
Standard margin (lux %): 2.54
Standard margin (U0 %): 9.65
Fixtures: 24
Spacing X: 2.5m, Spacing Y: 2.67m
```

Problems with raw output alone:

- A user who is not a lighting engineer cannot interpret what these numbers mean
- "Standard margin (lux %): 2.54" — is that bad? Good? Critical?
- No guidance on what to change to fix a failing result
- All options are presented equally with no quality ranking beyond pass/fail
- No cross-option comparison (is option 3 better than option 1 in ways beyond compliance?)
- No context: "this room type typically needs X" — the standard row tells you the minimum but not the engineering judgment

After: AI pipeline adds interpretation layer

The same result, after AI analysis:

```
Quality Score: 65 / 100   (Confidence: 90%)
Source: Gemini — account_1_free

Issues (2):
  HIGH   Average Illuminance (Em)
         Calculated lux is 2.6% below required 500 lux
         → Increase fixture count to 26 or use higher efficacy lamps

  MEDIUM Uniformity (U0)
         U0=0.54 is below required minimum of 0.60
         → Adjust fixture spacing or switch to wider-beam luminaire

Suggestions:
  • Consider SC triproof for better uniformity in this room footprint
  • Review mounting height — increasing from 3.2m to 3.5m would improve beam overlap
```

```
Summary:
  Design narrowly misses both lux and uniformity targets. Minor adjustments
  to fixture count or type will achieve full compliance with minimal cost impact.
```

Comparison table

Aspect	Before AI pipeline	After AI pipeline
Compliance signal	Pass/fail numbers	Scored 0–100 with color coding
Issue identification	None — user must interpret margins	Explicit issues with severity (HIGH/MEDIUM/LOW)
Fix guidance	None	Specific actionable suggestions per issue
Engineering context	None	AI applies domain knowledge about space types
Non-expert usability	Requires lighting knowledge to interpret	Self-explanatory for any user
Multi-option comparison	Side-by-side numbers only	Quality score enables ranking
Improvement over time	None — static results	Snapshot system tracks approved baselines
Calculation improvement feedback	None	Approved fixes become training examples

Where AI adds most value

High-value scenarios:

1. **Narrowly failing designs** — margins of 2–5% where the fix is simple (add 2 fixtures, change wattage). Raw output shows the gap; AI says what to do about it.
2. **Multiple failing criteria** — when both lux and U_0 fail, the AI prioritizes which to address first (high severity = fix this first).
3. **Non-standard room types** — a space with unusual dimensions where the standard reference provides the lux target but no practical guidance on achieving it.
4. **Client-facing reports** — the quality score and summary are directly quotable in a report without requiring the client to understand photometric notation.

5. **Batch analysis** — running the AI on all saved studies to identify which past projects had marginal compliance that might benefit from redesign.

Lower-value scenarios:

1. Clearly passing designs (quality score ≥ 85) — the AI confirmation is useful but adds less new information
2. Very simple single-room calculations with a single fixture option

---

Part 6 — How the Pipeline Improves Results Over Time

Current mechanism: snapshot baseline

Every approved AI analysis is saved as the snapshot. When Gemini is unavailable, the system returns the last approved analysis rather than an empty default. This means:

- Day 1: No snapshot → generic default response
- Day 5: After several approved analyses → snapshot reflects real project patterns
- Day 30: Snapshot captures common issues for your building type and fixture catalog

Future mechanism: local model fine-tuning

The architecture is designed to support this workflow:

1. Gemini analyzes a result → user approves the fix
2. The (payload, analysis) pair is stored as a training example
3. When enough examples accumulate, the local Ollama model can be fine-tuned on this data
4. The fine-tuned model learns patterns specific to your fixture catalog, room types, and standards
5. Over time, Ollama local quality approaches Gemini quality for your specific domain

This is possible because all the data stays local — the study JSONs, the approved analyses, and the model are all on your server.

Part 7 — API Reference

POST `/api/ai/analyze`

Analyze a saved study or inline payload.

Request:

```
{ "token": "bca62ed6b65842ca4efd0e2f527d1246" }
```

or inline:

```
{ "sides": [...], "height": 3.2, "results": [...], "standard_lighting": {...} }
```

Response:

```
{
  "status": "success",
  "source": "gemini:account_1_free",
  "confidence": 0.9,
  "quality_score": 70,
  "issues": [...],
  "suggestions": [...],
  "summary": "..."
}
```

POST `/api/ai/approve-fix`

Save the current analysis as the fallback snapshot.

Request:

```
{ "analysis": { ...the analysis object... }, "token": "optional_study_token" }
```

GET `/api/ai/status` *(admin only)*

Returns quota status for all configured accounts plus snapshot info.

PUT `/api/ai/account` *(admin only)*

Update an account key at runtime without restarting Flask.

Request:

```
{ "name": "account_2_free", "api_key": "AIzaSy...", "enabled": true }
```

Part 8 — Model Comparison

Model	Type	RPD (free)	Output quality	Speed	Token limit
`gemini-2.5-flash`	Gemini online	20	High	~5s	~40 tokens (truncates)
`gemini-3.1-flash-lite-preview`	Gemini online	500	High	~3s	Full response ✓
`llama3.2:3b`	Ollama local	Unlimited	Medium	~8s	Full response ✓
`gemma3:27b`	Ollama local	Unlimited	High	~45s	Full response ✓

Current recommended setup:

- `ollama_priority: true` — use Ollama (gemma3:27b) for all local development
- `model: gemini-3.1-flash-lite-preview` — used when Ollama unavailable or deployed online
- Free accounts as backup chain (accounts 1–4)
- Paid account (account 5) for production deployment

Part 9 — Security Notes

- `gemini_config.json` must be in `.gitignore` — it contains live API keys
- `gemini_snapshot.json` must be in `.gitignore` — may contain project data
- `/api/ai/status` and `/api/ai/account` are protected by `_admin_session_ok()` — same auth as admin dashboard
- `/api/ai/analyze` is public (same as `/calculate`) — it only returns analysis, never exposes keys
- Study token is validated against `[a-f0-9]{32}` pattern before file access
- Only lighting metrics are sent to Gemini — customer name, email, phone are never included in the prompt